

BidShield AI

AI-Powered GeM Bid Compliance Verification Platform

Smart India Hackathon — Prototype Blueprint & Build Plan

Problem Statement ID	26100
Problem Statement Title	AI-Powered Integrated Bid Compliance Verification Platform for GeM Procurement
Document purpose	Step 1 deliverable: complete UI/UX screen structure, folder structure and database schema, to be followed by module-by-module code build.
Companion artifact	bidshield-prototype.html — interactive, clickable demo (mock data, no live government API calls) rendered with a Three.js portal-network visualization.

Important framing for judges: AI is a decision-support layer only. Every screen in this blueprint keeps the qualify / disqualify decision with the human Procurement Officer, and every government-portal integration is a mock connector backed by a dummy dataset for demo purposes — matching the problem statement's own allowance for synthetic test data.

Contents: 1. System architecture · 2. Screen-by-screen UI/UX structure · 3. Folder structure · 4. Database schema · 5. API contract · 6. Compliance & risk-scoring logic · 7. Tech stack · 8. Demo script for judges · 9. Build roadmap

1. System architecture

The platform is organised as five layers: the Officer-facing dashboard, three functional modules (Tender, Bidder, Verification), a central AI Verification Engine, a set of mock government-portal connectors, and a Compliance Engine that produces the score, risk level and recommendation shown to the officer.

Layer	Component	Responsibility
Presentation	Web Dashboard (React / HTML+JS)	Login, tender creation, bidder management, compliance views, audit trail
Application modules	Tender Module · Bidder Module · Verification Module	CRUD for tenders/bidders, document upload, triggers verification runs
AI layer	AI Verification Engine (OCR → NLP → Entity Extraction → Rules/LLM)	Reads documents, extracts structured fields, flags inconsistencies
Integration layer	Mock connectors: GSTN, PAN, Udyam, MCA21, EPFO, ESIC, NSIC, Startup India, Blacklist Registry	Return dummy but realistic portal responses via REST endpoints
Decision layer	Compliance Engine → Risk Engine → Recommendation Engine	Weighted scoring, LOW/MEDIUM/HIGH classification, explainable recommendation
Governance	Audit Trail + Officer Approve/Reject	Immutable timestamped log; human keeps the final decision

1.1 Request flow

Officer → Web Dashboard → REST API (Flask) → Tender / Bidder / Verification modules → AI Verification Engine → mock portal connectors (GSTN, PAN, Udyam, MCA21, EPFO/ESIC ...) → Compliance Engine → Score + Risk + Discrepancies → Recommendation → back to Officer for the final Approve / Request Clarification / Reject decision, all written to the Audit Trail.

Step	What happens
1	Officer uploads the tender PDF/DOCX; AI extracts the Tender Compliance Matrix (turnover, GST, PAN, MSME, OEM, Make in India, certifications...).
2	Officer adds bidders with registration numbers (GSTIN, PAN, CIN, Udyam...) and supporting documents.
3	Verification pipeline runs: OCR → Text Extraction → Entity Extraction → Field Validation → Cross-Verification against mock portal APIs.
4	Compliance Engine applies weighted rules per requirement; Risk Engine checks for mismatches, missing docs, expired certs, blacklisting.
5	Recommendation Engine produces a plain-language verdict (Fully Compliant / Conditionally Compliant / Non-Compliant) with reasons and a suggested next action.
6	Officer reviews the compliance matrix, discrepancies and explainable-AI chain, then Approves, Requests Clarification, or Rejects. This decision — not the AI's score — is final.

2. Screen-by-screen UI/UX structure

Two roles are supported: **Procurement Officer** (day-to-day evaluation) and **Administrator** (users, rules, connectors, audit oversight). The screen list below is what the companion HTML prototype implements end-to-end with mock data.

Login

Role selector (Officer / Admin), credential fields. Hero visual: an animated Three.js network — a central AI-engine node with GSTN / PAN / Udyam / MCA21 / EPFO / ESIC / NSIC portal nodes orbiting and pulsing — establishing the multi-portal integration story before the officer even logs in.

Dashboard

Stat cards (active tenders, bids under review, high-risk count, pending review) and a recent-tenders table linking into each tender.

Tender creation

Form for tender ID, organisation, department, value, deadline, minimum turnover; a document dropzone that triggers 'AI extraction' of requirements; toggle switches for each statutory requirement (GST, PAN, Udyam, OEM, Make in India, EPFO, ESIC, Startup India, blacklisting) so the officer can confirm or override what the AI extracted.

Tender detail

Read-only Tender Compliance Matrix (what's required and from which source-of-truth) plus the bidder list for that tender, each row showing GSTIN, running compliance score and risk band.

Bidder verification pipeline

A horizontal stepper — OCR → Text Extraction → Entity Extraction → Field Validation → Cross-Verification → Compliance Engine — that animates as it runs, with a live mock-API log panel showing the JSON calls/responses to each government connector.

Compliance score & matrix

A circular score gauge (0-100) coloured by risk band, a weighted-scoring explainer, and a full requirement-by-requirement matrix (Requirement / Source / Status / Evidence) with PASS / WARNING / FAIL badges.

Discrepancy detection

Side-by-side comparison of conflicting values pulled from different sources (e.g. GST legal name vs MCA21 legal name) with an AI confidence percentage and a 'manual verification recommended' action.

Risk indicators

A compact pill list of flags (GSTIN mismatch, OEM authorization unclear, address mismatch, blacklisting found, etc.), colour-coded red / amber / green.

Explainable AI chain

A vertical reasoning chain that shows *why* a requirement passed or failed step by step, rather than a single opaque verdict — this is the differentiator judges respond to.

AI recommendation & decision

Plain-language verdict with bullet reasons and a suggested next action, followed by three buttons — Approve / Request Clarification / Reject — that make clear the AI recommends but the officer decides.

Audit trail

Chronological, timestamped, immutable log of every system and officer action from tender creation through to the final decision.

Admin — portal connectors

A settings screen listing each mock connector (GSTN, PAN, Udyam, MCA21, EPFO, ESIC, Blacklist Registry), its endpoint and connection status — this is where real credentials would be wired in post-hackathon.

3. Folder structure

A conventional Flask + React split keeps the AI/rules logic testable independently of the UI. This is the structure to scaffold before writing feature code.

```
bidshield-ai/
|-- backend/
|   |-- app/
|   |   |-- __init__.py
|   |   |-- config.py
|   |   |-- models/                # SQLAlchemy models (see Section 4)
|   |   |   |-- user.py
|   |   |   |-- tender.py
|   |   |   |-- bidder.py
|   |   |   |-- document.py
|   |   |   |-- verification.py
|   |   |   |-- compliance.py
|   |   |   |-- audit.py
|   |   |-- routes/                # Flask blueprints (see Section 5)
|   |   |   |-- auth.py
|   |   |   |-- tenders.py
|   |   |   |-- bidders.py
|   |   |   |-- documents.py
|   |   |   |-- verification.py
|   |   |   |-- reports.py
|   |   |-- connectors/            # mock government-portal connectors
|   |   |   |-- gstn.py
|   |   |   |-- pan.py
|   |   |   |-- udyam.py
|   |   |   |-- mca21.py
|   |   |   |-- epfo_esic.py
|   |   |   |-- startup_india.py
|   |   |   |-- nsic.py
|   |   |   |-- oem.py
|   |   |   |-- blacklist.py
|   |   |-- ai_engine/
|   |   |   |-- ocr.py              # document OCR
|   |   |   |-- nlp_extract.py      # tender-requirement extraction
|   |   |   |-- entity_extract.py   # bidder-document field extraction
|   |   |   |-- cross_verify.py     # calls connectors, diff-checks fields
|   |   |   |-- explain.py          # builds the explainable-AI reasoning chain
|   |   |-- compliance_engine/
|   |   |   |-- rules.py            # requirement PASS/WARN/FAIL rules
|   |   |   |-- scoring.py          # weighted scoring (Section 6)
|   |   |   |-- risk.py             # LOW/MEDIUM/HIGH classification
|   |   |-- dummy_data/            # seed tenders/bidders/portal responses
|   |   |-- utils/
|   |-- tests/
|   |-- requirements.txt
|   |-- run.py
|-- frontend/
|   |-- src/
|   |   |-- pages/                 # one file per screen in Section 2
|   |   |-- components/           # ScoreGauge, ComplianceMatrix, PipelineStepper,
|   |   |                                   # DiscrepancyCard, RiskPillList, ExplainChain, AuditItem
|   |   |-- api/                  # REST client
```

```

|   |   |-- state/                # auth + app state
|   |   |-- App.jsx
|   |-- package.json
|-- docs/
|   |-- architecture.md
|   |-- api-contract.md
|   |-- demo-script.md

```

4. Database schema

SQLite is sufficient for the hackathon demo; the model maps directly onto PostgreSQL for a production build. Primary/foreign keys are implied by the `id` / `*_id` naming.

users

Column	Type	Notes
id	INTEGER PK	Auto-increment
name	VARCHAR	
email	VARCHAR	Unique, login identifier
password_hash	VARCHAR	bcrypt hash
role	VARCHAR	'officer' or 'admin'
created_at	DATETIME	

tenders

Column	Type	Notes
id	INTEGER PK	
tender_number	VARCHAR	e.g. CPCL/2026/PROC/001
title	VARCHAR	
organization	VARCHAR	
department	VARCHAR	
value	DECIMAL	Tender value in INR
min_turnover	DECIMAL	
deadline	DATE	
requirements_json	JSON	AI-extracted / officer-confirmed requirement toggles
status	VARCHAR	draft / reviewing / completed
created_by	INTEGER FK	→ users.id
created_at	DATETIME	

bidders

Column	Type	Notes
id	INTEGER PK	
tender_id	INTEGER FK	→ tenders.id
company_name	VARCHAR	
gst_in	VARCHAR	
pan	VARCHAR	
cin	VARCHAR	
udyam_no	VARCHAR	Nullable
startup_id	VARCHAR	Nullable
nsic_no	VARCHAR	Nullable

documents

Column	Type	Notes
id	INTEGER PK	
bidder_id	INTEGER FK	→ bidders.id
document_type	VARCHAR	GST Certificate, PAN, Udyam, ITR, OEM Authorization...
filename	VARCHAR	
upload_date	DATETIME	
verification_status	VARCHAR	pending / pass / warning / fail
confidence	FLOAT	OCR/entity-extraction confidence 0-1

verification_results

Column	Type	Notes
id	INTEGER PK	
bidder_id	INTEGER FK	→ bidders.id
verification_type	VARCHAR	gst / pan / udyam / mca / epfo / esic / blacklist ...
source	VARCHAR	Connector name
status	VARCHAR	pass / warning / fail
result	JSON	Raw mock-connector response
confidence	FLOAT	
timestamp	DATETIME	

compliance_results

Column	Type	Notes
id	INTEGER PK	
bidder_id	INTEGER FK	→ bidders.id
requirement	VARCHAR	e.g. 'OEM Authorization'
status	VARCHAR	pass / warning / fail
score	FLOAT	Weighted contribution to overall score
reason	TEXT	Explainable-AI reasoning text
evidence	VARCHAR	Document / source reference

audit_logs

Column	Type	Notes
id	INTEGER PK	
user_id	INTEGER FK	→ users.id, nullable for system events
action	VARCHAR	e.g. 'GST Verification Passed'
entity	VARCHAR	tender / bidder / document reference
timestamp	DATETIME	
details	TEXT	

5. API contract

5.1 Application endpoints

Method	Endpoint	Purpose
POST	/api/auth/login	Authenticate officer/admin, returns JWT
POST	/api/tenders	Create a tender
GET	/api/tenders	List tenders
GET	/api/tenders/:id	Tender detail incl. compliance matrix
POST	/api/tenders/:id/bidders	Add a bidder to a tender
POST	/api/bidders/:id/documents	Upload bidder documents
POST	/api/bidders/:id/verify	Run the verification pipeline
GET	/api/bidders/:id/compliance	Compliance matrix + score
GET	/api/bidders/:id/risk	Risk indicators
GET	/api/bidders/:id/recommendation	AI recommendation
GET	/api/bidders/:id/audit	Bidder-scoped audit trail
POST	/api/bidders/:id/decision	Officer records approve / clarify / reject
GET	/api/reports/:id	Generate downloadable PDF compliance report

5.2 Mock government-portal connectors

Each connector is a small Flask blueprint returning a realistic but synthetic payload, so the demo never touches a real government system.

Endpoint	Request	Mock response
/api/verify/gst	{"gstn": "19ABCDE1234F1Z5"}	{"status": "ACTIVE", "legal_name": "ABC Technologies Pvt Ltd", "filing_status": "COMPLIANT", "last_return": "2026-08-31"}
/api/verify/pan	{"pan": "ABCDE1234F"}	{"status": "VALID", "name_match": true}
/api/verify/udyam	{"udyam_no": "..."}	{"status": "REGISTERED", "category": "Small"}
/api/verify/mca	{"cin": "..."}	{"legal_name": "...", "status": "ACTIVE"}
/api/verify/epfo	{"establishment_id": "..."}	{"status": "COMPLIANT"}
/api/verify/esic	{"esic_no": "..."}	{"status": "COMPLIANT"}
/api/verify/blacklist	{"pan" or "cin": "..."}	{"listed": false}

6. Compliance & risk-scoring logic

6.1 Weighted scoring model

Requirement	Weight
GST	10
PAN	10
Udyam	5
IT Compliance	15
EPFO	5
ESIC	5
OEM	15
Make in India	10
Experience	15
Blacklisting	10

6.2 Rule examples

IF GST required AND GST status = ACTIVE AND GST returns = COMPLIANT
THEN GST compliance = PASS

IF minimum turnover = ■10 Cr AND bidder turnover >= ■10 Cr
THEN financial eligibility = PASS ELSE FAIL

IF OEM authorization required AND document exists AND OEM = registered manufacturer
THEN OEM compliance = PASS ELSE FAIL

6.3 Risk bands

Band	Score range	Meaning
LOW	85 – 100	Fully compliant, minimal officer review needed
MEDIUM	60 – 84	Conditionally compliant, targeted manual checks required
HIGH	Below 60	Non-compliant / disqualifying issues found

Every score screen carries the disclaimer: "AI-generated risk assessment. Final qualification/disqualification decision must be made by the Procurement Officer." This directly satisfies the problem statement's requirement that the AI remain a decision-support tool.

7. Technology stack

Layer	Choice
Frontend	React + Tailwind CSS (or plain HTML/CSS/JS for the fastest prototype path); Three.js for the portal-network visualization; Chart.js for score/trend charts.
Backend	Python + Flask, REST API.
AI / NLP	Python — OCR (pytesseract), NLP requirement extraction, document classification, entity extraction, rule engine (optionally LLM-assisted for free-text clauses).
Database	SQLite for the prototype; PostgreSQL for production.
File storage	Local disk for the demo; object storage (S3-compatible) in production.
Auth	JWT / session-based authentication, role-based access (Officer vs Admin).
Charts	Chart.js for gauges/trends; native SVG for the compliance score ring.

8. Demo script for judges (≈5 minutes)

- 1. Open on the Login screen — narrate the Three.js visual: “This represents the eight-plus government portals GeM bid verification currently has to check manually.”
- 2. Sign in as Procurement Officer → Dashboard: point out active tenders, bids under review, and the high-risk counter updating live.
- 3. Open tender CPCL/2026/PROC/001 → show the AI-extracted compliance matrix pulled from the uploaded tender PDF.
- 4. Open Bidder 1 — ABC Engineering Pvt Ltd → click Run Verification → let the pipeline animate (OCR → Extraction → Cross-Verification) with the mock-API log visible → land on a 96/100, LOW risk, fully compliant result.
- 5. Open Bidder 2 — XYZ Industrial Solutions → run verification → show the OEM authorization discrepancy (manufacturer-name mismatch, 87% confidence) and the Conditionally Compliant recommendation.
- 6. Open Bidder 3 — PQR Enterprises → run verification → show the blacklisting hit and entity-name mismatch → Non-Compliant, recommend disqualification.
- 7. Walk through the Explainable-AI chain for one failing requirement so judges see the reasoning, not just a verdict.
- 8. Click Approve / Reject to show the officer keeps the final call, then open Audit Trail to show the immutable, timestamped record of the whole session.

9. Build roadmap after this blueprint

- Freeze this architecture, schema and API contract (this document) as the shared reference.
- Scaffold the folder structure in Section 3 and stand up the Flask app skeleton + SQLite models from Section 4.

- Build the mock connectors (Section 5.2) against a small dummy dataset so cross-verification has something real to call.
- Wire the compliance/risk/recommendation engines (Section 6) as pure functions so they're independently testable.
- Build the screens in Section 2 against the live API, replacing the prototype's hard-coded mock data one screen at a time.
- Layer in the OCR/NLP pipeline for tender-document and bidder-document extraction once the rules engine is stable.
- Finish with the PDF compliance report generator and the audit-trail export, then rehearse the demo script in Section 8.